

Rapport : Système RAG Agentique

Master Ingénierie Informatique, Big Data et Cloud Computing

Youssef Belahrach

1 Démarche Suivie et Architecture

Ce projet vise à concevoir un système RAG Agentique appliqué au domaine de l'architecture Cloud et DevOps.

Pour s'affranchir du côté "boîte noire" de la méthode `create_agent` de LangChain, le système a été développé avec **LangGraph**. Cela permet de modéliser l'agent sous forme de graphe d'états et d'offrir un contrôle précis sur le flux d'exécution et la mémoire.

2 Technologies Utilisées

Le projet a été implémenté sous macOS en utilisant le gestionnaire d'environnement `uv` pour garantir une installation ultra-rapide et déterministe des dépendances. La base vectorielle repose sur **ChromaDB**, couplée à l'API **Tavily** pour les recherches externes ainsi que **OpenAI (GPT-4o-mini et Text-Embedding-3-small)** comme modèles de langage et d'embeddings.

3 Fonctionnement du Système

L'architecture s'articule autour d'un état partagé (**AgentState**) qui transite à travers quatre étapes clés :

1. **Retrieve** : Extraction des documents pertinents depuis la base vectorielle locale.
2. **Grade** : Un LLM évalue les documents récupérés. Si les informations sont insuffisantes, il lève un drapeau signalant le besoin d'une recherche externe.
3. **Web Search (Conditionnel)** : Activé uniquement si les documents locaux sont incomplets. L'agent interroge le web (Tavily) pour compléter son contexte.
4. **Generate** : Synthèse de la réponse finale basée strictement sur le contexte validé lors des étapes précédentes.

4 Résultats de l'Évaluation

Le système a été testé sur 20 questions (10 simples et 10 complexes).

Analyse des résultats obtenus : L'agent optimise intelligemment ses ressources. Pour les requêtes simples, il répond rapidement via la base locale. Pour les requêtes complexes (ex : actualités ou architectures multi-outils absentes de la base), il détecte correctement ses limites et s'enrichit via le web, prouvant ainsi son autonomie décisionnelle.

5 Limites et Pistes d'Amélioration

5.1 Limites actuelles

- **Latence** : Le processus d'évaluation impose un appel LLM supplémentaire avant la génération, ralentissant le temps de réponse global.
- **Dépendance aux API** : Les questions complexes dépendent de la disponibilité de l'API Tavily et génèrent un coût (tokens).
- **Mémoire** : L'agent traite chaque requête de manière isolée, sans conserver d'historique conversationnel entre deux questions.

5.2 Pistes d'amélioration

- **Modèles Locaux** : Remplacer les API propriétaires par des modèles open-source exécutés en local (comme Llama 3.2 via Ollama) pour réduire les coûts et garantir la confidentialité des données Cloud.
- **Self-Correction** : Ajouter un nœud d'audit post-génération pour vérifier l'absence d'hallucinations avant d'afficher la réponse finale.
- **Mémoire à long terme** : Intégrer le `MemorySaver` natif de LangGraph pour offrir une expérience conversationnelle fluide avec suivi de l'historique utilisateur.